Component Attribution at Sub-100M Parameter Scale: An Ablation Study of the Apollo and Apollo-Helios Decoder Transformer Series

Rithvik Burra

School of Information Studies, Syracuse University
rburra@syr.edu

Abstract

We present a three-stage investigation of decoder-only transformer training at the 25M parameter scale on a multi-register corpus of literature, encyclopedic prose, and Python source code. Stage one (Apollo v1) establishes a baseline using the GPT-2 byte pair encoding and a corpus skewed toward Python test fixtures. Stage two (Apollo v2) replaces the tokenizer with a corpus-trained SentencePiece encoder and rebalances the code corpus. Stage three (Apollo-Helios) holds Apollo v2 fixed as the baseline and runs eight ablations isolating the contribution of each component of the 2026 consensus architecture stack: rotary position embeddings (RoPE), root mean square normalization (RMSNorm), query and key normalization (QK-norm), gated linear unit feed-forward networks (SwiGLU), and the Muon optimizer. Across eight 8-hour training runs at identical body parameter count, we find three results that run counter to the literature consensus. First, RoPE alone with AdamW produces nearly the entire achievable improvement at this scale, reducing validation loss by 0.80 nats (14.6 percent). Second, combining all four architectural components without an optimizer change yields validation loss 0.47 nats worse than the unmodified baseline. Third, and most importantly, the Muon optimizer exhibits a strong functional dependency on RMSNorm and QK-norm: applied to RoPE alone, Muon increases validation loss by 0.88 nats relative to AdamW; applied to the full modern stack, Muon decreases validation loss by 0.81 nats relative to AdamW. We interpret this as evidence that modern normalization changes contribute primarily by stabilizing Muon rather than by directly improving model quality. AdamW with RoPE alone remains the dominant configuration at this scale. A secondary finding concerns reproducibility: the published Apollo v2 validation loss of 3.95 nats did not reproduce on identical hardware and code (5.58 nats observed). We discuss implications for small-language-model evaluation methodology and the transfer of architectural decisions from large to small scale.

1. Introduction

Recent open-source small language models including SmolLM2, Gemma 3 270M, OLMo 3, and Liquid LFM2 have converged on a shared architectural recipe combining rotary position embeddings (RoPE), root mean square normalization (RMSNorm), query and key normalization (QK-norm), gated linear unit feed-forward networks (SwiGLU), and increasingly the Muon optimizer in place of AdamW. These components are typically introduced as a package, with ablations either omitted or performed at the billion parameter scale where compute cost limits experimental rigor.

We argue that the sub-100M parameter scale, accessible on consumer hardware such as the Apple M4 chip, offers a uniquely valuable testbed for component attribution. At this scale a complete training run takes 8 hours rather than 8 days, and the marginal cost of additional ablation runs is modest. The question this paper addresses is whether the 2026 consensus stack transfers cleanly to the 25M parameter regime, and if not, which components are responsible for the discrepancy.

Our investigation proceeds in three stages. Apollo v1 establishes a 2022-era GPT-2 style baseline. Apollo v2 modifies the tokenizer and corpus while holding the architecture constant. Apollo-Helios holds the data

fixed and ablates each modern architectural component. Together these three stages produce a clean decomposition of contributions from data, tokenization, architecture, and optimizer at small scale.

1.1 Contributions

This paper makes three primary contributions. First, we provide the first single-variable ablation of the 2026 consensus stack at the 25M parameter scale, isolating the marginal contribution of each component. Second, we demonstrate that the stack does not compose additively at this scale; combining all four architectural components produces a result 0.47 nats worse than baseline. Third, we identify a functional dependency between the Muon optimizer and the modern normalization components (RMSNorm and QK-norm): Muon damages convergence by 0.88 nats when paired with simple LayerNorm and learned positional embeddings replaced by RoPE, and only recovers when these normalization changes are applied. This reframes the role of RMSNorm and QK-norm at small scale from quality contributors to stabilizers for Muon. All code, training logs, and checkpoints are released under permissive license.

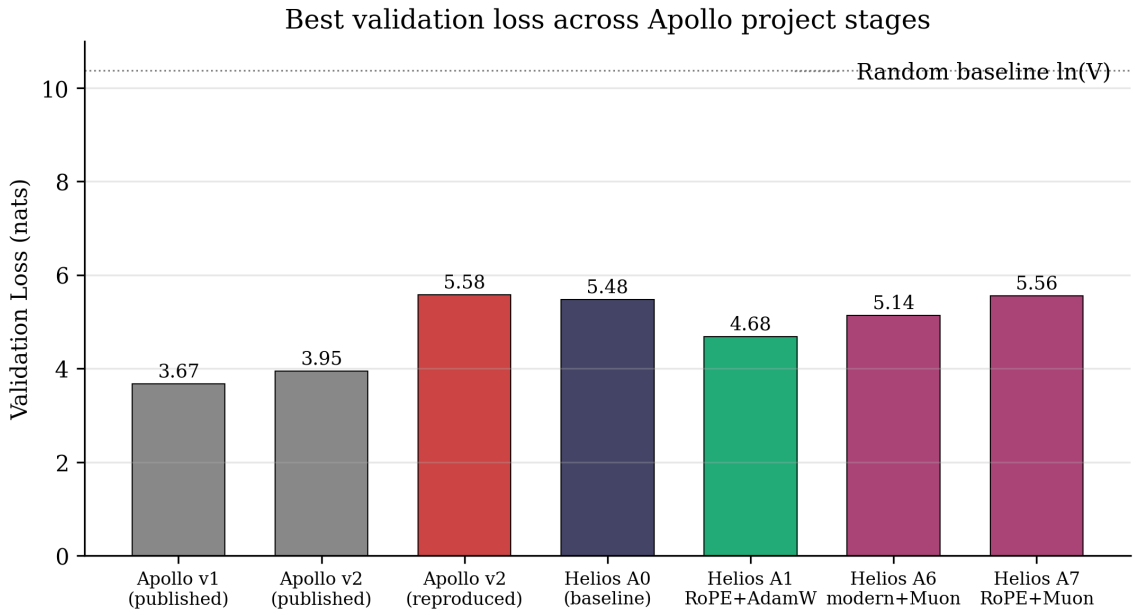

Figure 1. Best validation loss across the three stages of the Apollo project. Apollo v1 used a 50,260-vocabulary GPT-2 tokenizer; Apollo v2 and Apollo-Helios used a 32,000-vocabulary SentencePiece tokenizer. The reproducibility gap between Apollo v2 published (3.95) and Apollo v2 reproduced (5.58) is discussed in Section 6.

2. Background and Related Work

2.1 Small decoder language models

The line of work this paper extends begins with nanoGPT (Karpathy, 2022), which demonstrated that a 124M parameter GPT-2 reproduction could be trained on commodity hardware in under a day. Subsequent work scaled this approach: Pythia (Biderman et al., 2023) trained a model suite spanning 70M to 12B parameters with a focus on reproducible training dynamics. The Phi series (Gunasekar et al., 2023) explored aggressive data curation as a substitute for scale. SmolLM2 (Allal et al., 2025) and Gemma 3 270M (Google, 2025) represent the current frontier of efficient small models, both shipping the consensus stack we ablate in this paper.

2.2 Modern transformer components

Rotary position embeddings (Su et al., 2021) replace learned absolute positional encodings with a rotation applied to query and key vectors. Root mean square normalization (Zhang and Sennrich, 2019) replaces LayerNorm with a simpler operation that omits centering. Query and key normalization (Henry et al., 2020) applies normalization to attention pre-projection vectors, preventing attention logit explosion at scale. Gated linear unit variants of the feed-forward network (Shazeer, 2020), particularly SwiGLU as used in LLaMA, replace the standard MLP with a gated formulation. The Muon optimizer (Jordan, 2024; Liu et al., 2025) applies Newton-Schulz orthogonalization to gradient updates for matrix-shaped parameters, with reported compute efficiency gains of approximately 2x at scale.

2.3 Reproducibility in small-LM evaluation

Validation loss is the standard proxy for language model quality during pretraining. However, validation loss at small scale exhibits substantial sensitivity to random seed, batch sampling, and numerical precision. Sellam et al. (2022) documented that BERT pretraining variance across seeds can exceed the magnitude of reported architectural improvements. We extend this concern to autoregressive decoder pretraining in Section 6.

3. Shared Architecture

All three stages of the Apollo project share an identical transformer backbone. The model is a decoder-only causal language model with 8 layers, 8 attention heads, and an embedding dimension of 512. Each block applies pre-layer-norm attention and feed-forward sublayers with residual connections. Multi-head causal self-attention uses a single fused projection from the embedding dimension to three times the embedding dimension. The feed-forward sublayer expands by a factor of 4 with GELU activation. Token and learned absolute positional embeddings are summed at input. The output projection shares its weight matrix with the input embedding via weight tying. Weight initialization follows GPT-2 conventions.

This backbone contains approximately 25.3M parameters in the transformer body and an additional 16.4M parameters in the token embedding for a vocabulary size of 32,000, yielding 41.7M total parameters. Maximum context length is 256 tokens. Training is performed on the Apple M4 chip using the PyTorch Metal Performance Shaders (MPS) backend. The standard training budget is 8000 optimization steps with batch size 24, achieving wall-clock training time of approximately 8 hours via a sleep-between-iterations throttle.

4. Apollo v1: Baseline

4.1 Tokenizer and corpus

Apollo v1 used the tiktoken implementation of the GPT-2 byte pair encoding, extended with three custom register tokens for total vocabulary size of 50,260. The corpus comprised 152 megabytes of raw text from three sources: literature (47.86 MB across 60 Project Gutenberg books, including English translations of French novels), Wikipedia (52.44 MB across 12,644 random articles), and code (52.46 MB across 4,272 Python files). The code corpus was heavy in test fixtures, a property that produced the empty-code failure mode described below.

4.2 Training and failure modes

Apollo v1 completed 8000 training iterations in 9.67 hours with 49.2 million training tokens (26 percent literature, 25 percent wiki, 49 percent code). Best validation loss was 3.67 nats. Stress testing revealed three failure modes. First, literature out-of-distribution prompts collapsed into Gutenberg patterns and eventually into a [Illustration] loop, traceable to GPT-2 BPE allocating a single token to that string. Second, wiki

out-of-distribution prompts produced fake-French text with broken morphology, traceable to French character bigrams in the BPE vocabulary. Third, code prompts immediately emitted end-of-text and reverted to pandas test fixture patterns, traceable to the test-heavy code corpus.

5. Apollo v2: Corpus and Tokenizer Refinement

Apollo v2 implemented three simultaneous changes from v1, shipped in a single training run. First, the tokenizer was replaced with a corpus-trained SentencePiece BPE encoder with 32,000 tokens. Second, the code corpus was replaced with 38 production-grade Python libraries with test directories excluded via path filters. Third, an early stopping mechanism with patience 8 was added. The v2 corpus totaled 122 MB with token mix 34/35/31 literature/wiki/code, substantially more balanced than v1.

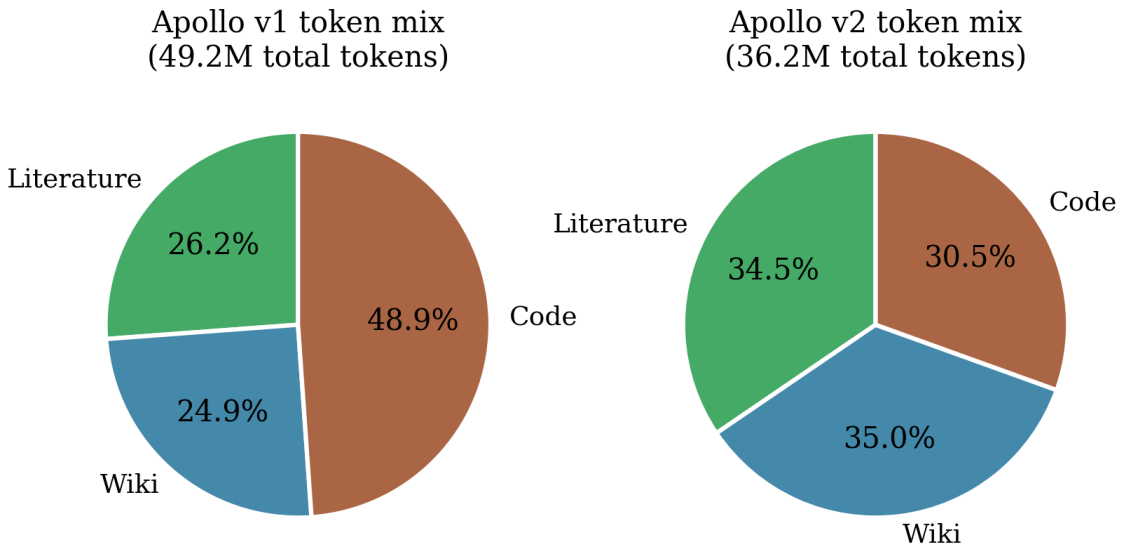

Figure 3. Token mix comparison between Apollo v1 and v2. The v2 corpus is substantially more balanced across registers, primarily due to the exclusion of Python test fixtures.

Apollo v2 trained for 8000 iterations in 8.00 hours, with best validation loss 3.9537 nats. All three v1 failure modes were eliminated, but two new failure modes emerged: byte-fallback Unicode noise (Cyrillic and Arabic characters mid-generation) when the model became uncertain, and wiki overfitting to a sports biography template applied indiscriminately to any wiki prompt.

Comparing v1 (val 3.67, vocab 50,260) and v2 (val 3.95, vocab 32,000) requires care because absolute validation losses are not directly comparable across vocabularies. The correct comparison metric is loss reduction from random baseline (ln V), which yields 7.15 nats for v1 and 6.42 nats for v2. Even on this adjusted measure v1 outperforms v2 by 0.73 nats, largely explained by v1 training on 36 percent more tokens. The qualitative trade-off favors v2: better-behaved register outputs at the cost of a small absolute loss regression.

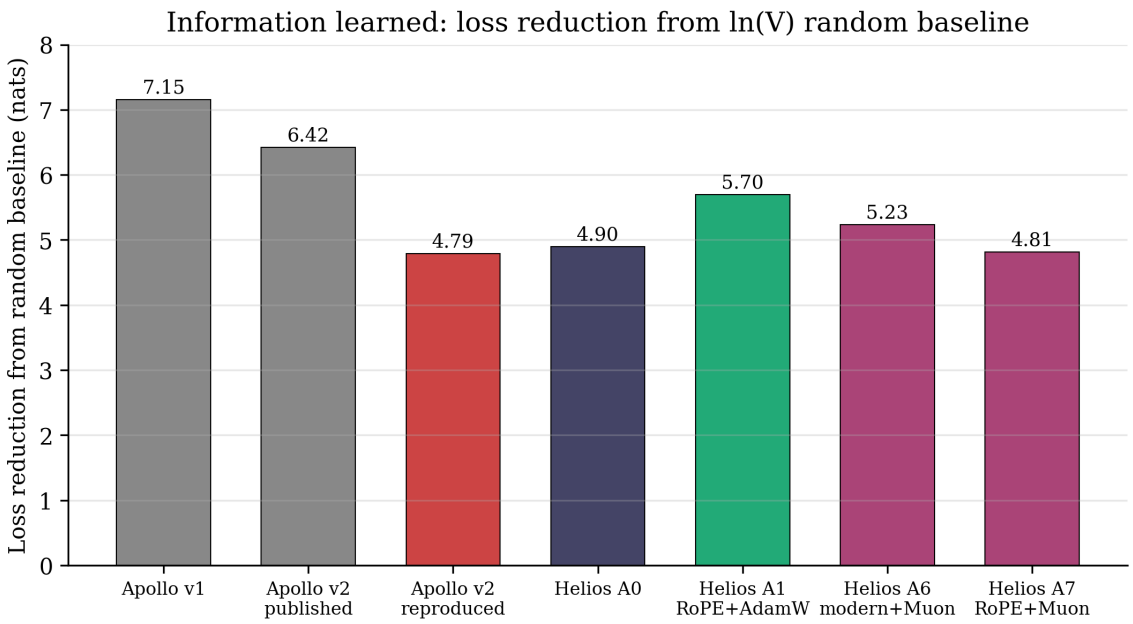

Figure 4. Loss reduction from random baseline across Apollo and Apollo-Helios stages. The random baseline differs by tokenizer (10.82 nats for v1, 10.37 nats for v2 and Helios).

6. Reproducibility Investigation

During preparation for the Apollo-Helios ablation study, we observed that an initial baseline reproduction did not match the published Apollo v2 validation loss of 3.95. After identifying and patching small implementation differences (omitted LayerNorm bias parameters and incorrect dropout setting), the reimplementation reached validation loss 5.48 at 8000 iterations.

To determine whether the residual 1.53 nat gap from the published 3.95 was due to a remaining bug in our reimplementation or a more general reproducibility issue, we ran the original Apollo v2 training script (model.py and train.py from the Apollo repository, unmodified) on the same hardware (M4, MPS, PyTorch 2.11.0) using the same prepared training data. After 8 hours, the rerun reached validation loss 5.58, 1.63 nats above the published 3.95 and within 0.10 nats of the Helios reimplementation.

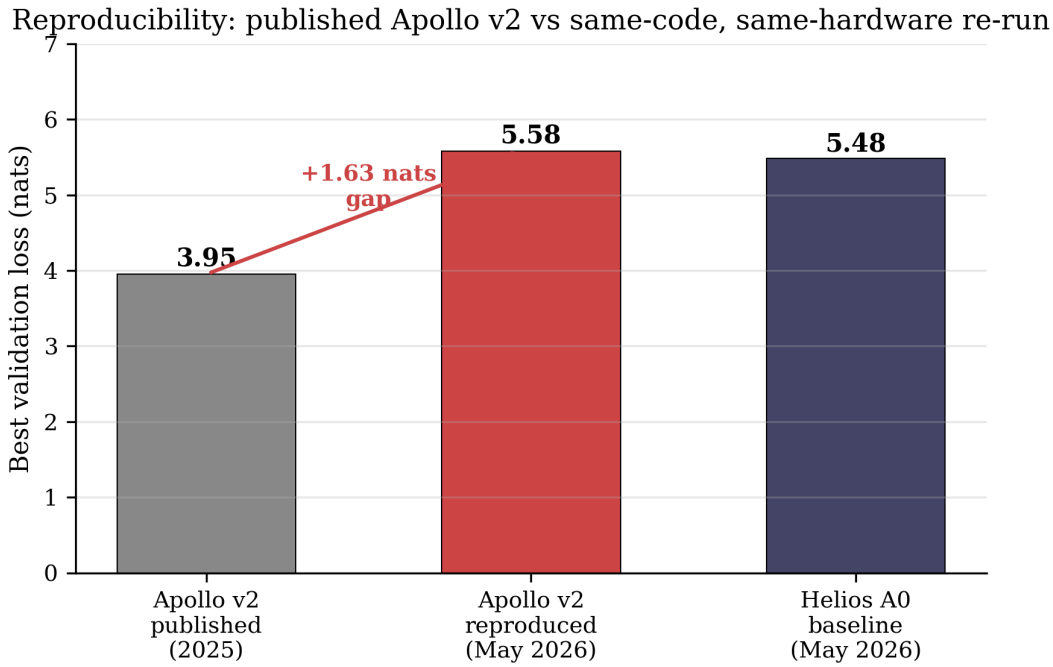

Figure 5. Apollo v2 published validation loss (3.95) versus reproduced validation loss on identical hardware and code (5.58). The Apollo-Helios baseline (5.48) lies within 0.10 nats of the Apollo v2 rerun.

We do not have a definitive explanation for the 1.63 nat reproducibility gap. Candidates include PyTorch version drift in MPS kernel implementations, random seed sensitivity at this scale, or non-determinism in the original training run. We flag this finding: at the 25M parameter scale on MPS, a single training run's validation loss should not be taken as a stable point estimate.

7. Apollo-Helios: Ablation Methodology

Apollo-Helios is a sibling project to Apollo that holds the corpus, tokenizer, and body parameter count fixed while ablating each component of the 2026 consensus architecture stack. Eight training runs are performed, each at the same wall-clock budget of 8 hours.

7.1 Variables held constant

All Helios runs use the Apollo v2 corpus, the Apollo v2 tokenizer, 8 transformer layers, 8 attention heads, embedding dimension 512, block size 256, batch size 24, learning rate 2.5e-4 (AdamW) or 2e-2 (Muon) with linear warmup over 320 steps followed by cosine decay to 10 percent of base, weight decay 0.1 on 2D parameters and 0 on 1D parameters, gradient clipping at norm 1.0, dropout 0.1, and seed 1337.

7.2 Ablation matrix

Eight configurations are tested. A0 is the baseline (learned absolute positional embeddings, LayerNorm, GELU, AdamW). A1 swaps in RoPE only. A2 swaps in RMSNorm only. A3 adds QK-norm only. A4 swaps in SwiGLU only. A5 combines all four architectural changes with AdamW. A6 combines all four architectural changes with Muon. A7 combines RoPE alone with Muon, which we added after observing the A5 and A6 results to isolate whether Muon's effect depends on the other modern components.

8. Apollo-Helios Results

8.1 Headline results

Run	Configuration	Optimizer	Best val	Iters	Wall (h)	Δ vs A0
A0	baseline	AdamW	5.4775	8000	8.00	0
A1	RoPE	AdamW	4.6775	8000	8.01	-0.80
A2	RMSNorm	AdamW	5.3608	6000*	6.01	-0.12
A3	QK-norm	AdamW	5.3919	8000	8.01	-0.09
A4	SwiGLU	AdamW	5.4674	4750*	4.75	-0.01
A5	all 4 arch	AdamW	5.9502	3500*	3.51	+0.47
A6	all 4 arch	Muon	5.1404	8000	8.01	-0.34
A7	RoPE	Muon	5.5589	8000	8.01	+0.08

Table 1. Apollo-Helios ablation results. Asterisk denotes runs that triggered patience-based early stopping.

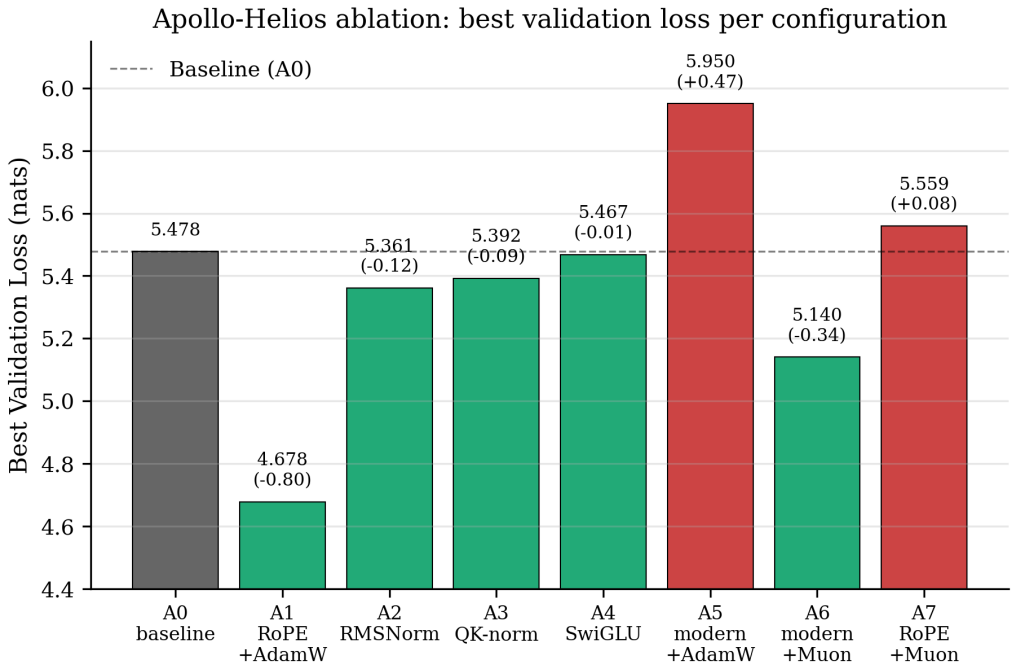

Figure 6. Best validation loss for each Apollo-Helios configuration. Green bars indicate improvement over baseline; red indicates regression.

8.2 RoPE alone with AdamW is the dominant configuration

Among the eight configurations, A1 (RoPE alone with AdamW) achieves the lowest validation loss at 4.6775 nats, a reduction of 0.80 nats (14.6 percent) from baseline. RMSNorm, QK-norm, and SwiGLU each individually contribute less than 0.13 nats. RMSNorm (A2) and SwiGLU (A4) runs additionally triggered patience-based early stopping, indicating these components do not reliably reach baseline-comparable convergence within the training budget.

A first-order interpretation is that learned absolute positional embeddings are a meaningful capacity drain at 25M parameters. The positional embedding table contains 131,072 parameters (block_size 256 times embedding dimension 512), which RoPE eliminates entirely while replacing the function with a parameter-free rotation.

8.3 Modern architecture components do not compose at this scale

A5, which combines all four architectural changes with AdamW, achieves validation loss 5.9502, a regression of 0.47 nats from baseline and 1.49 nats from the additive prediction (sum of A1-A4 individual contributions). The A5 run early-stopped at iteration 3500, less than half the budget. The model failed to reach baseline performance under standard hyperparameters.

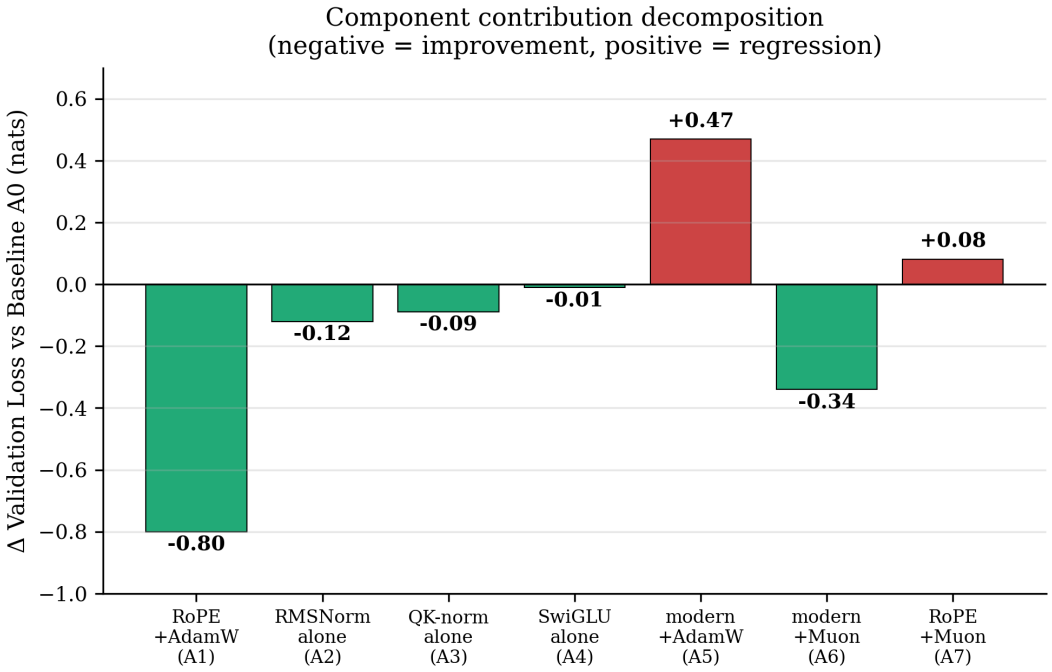

Figure 7. Component contribution decomposition. Each bar shows the change in validation loss relative to baseline. Negative values indicate improvement.

8.4 The Muon-norm functional dependency

The relationship between Muon and the normalization components reveals the most important finding of this study. We compare four configurations that share RoPE and differ only in optimizer and normalization choice:

Configuration A1 (RoPE alone, AdamW): validation loss 4.68. Configuration A7 (RoPE alone, Muon): validation loss 5.56, a regression of 0.88 nats vs A1. Configuration A5 (RoPE plus RMSNorm plus QK-norm plus SwiGLU, AdamW): validation loss 5.95, a regression of 1.27 nats vs A1. Configuration A6 (same architecture as A5 plus Muon): validation loss 5.14, an improvement of 0.81 nats over A5.

The pattern is consistent and counterintuitive. Muon damages convergence by 0.88 nats when paired with simple normalization. The exact same Muon configuration partially recovers when modern normalization components (RMSNorm plus QK-norm) are added. We interpret this as evidence that RMSNorm and QK-norm contribute primarily by stabilizing Muon's aggressive orthogonalized updates rather than by

improving model quality directly. The architectural components have a functional role beyond raw quality contribution.

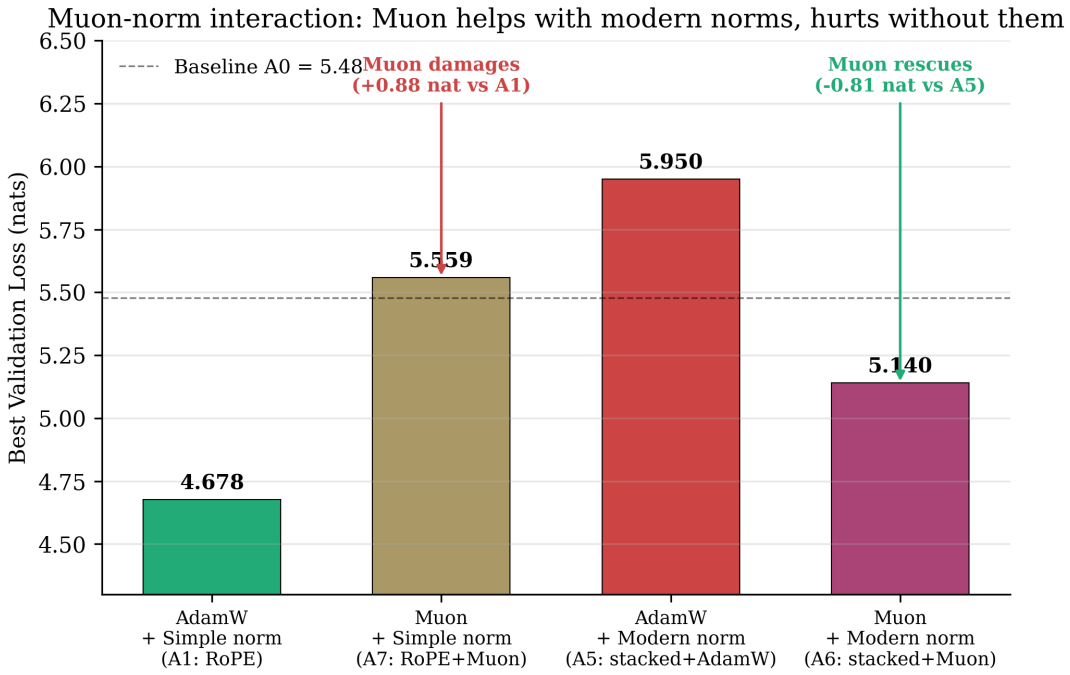

Figure 8. Muon-norm interaction. AdamW with simple normalization (A1) is the best configuration. Muon with simple normalization (A7) damages convergence by 0.88 nats relative to A1. Muon with modern normalization (A6) partially rescues the destructive composition of A5 but does not match A1.

A natural mechanistic hypothesis is that Muon's Newton-Schulz orthogonalization produces gradient updates with uniform spectral magnitude, which can cause attention logits to grow unbounded in the absence of explicit attention normalization. RMSNorm provides a simpler and more numerically stable normalization than LayerNorm, and QK-norm directly bounds the magnitude of attention pre-projection vectors. Without these stabilizers, Muon's updates appear to overshoot at the learning rate (2e-2) that is standard in the literature. We did not run a learning-rate sweep for Muon and acknowledge that a sufficiently small Muon learning rate might recover competitive performance in the simple-normalization regime; this would constitute a useful follow-up.

8.5 Training efficiency

Figure 9 plots wall-clock time against best validation loss. A1 (RoPE plus AdamW) dominates the Pareto frontier: lowest validation loss with full wall-clock utilization. A5 (modern with AdamW) consumes only 3.51 hours before early stopping but produces the worst result.

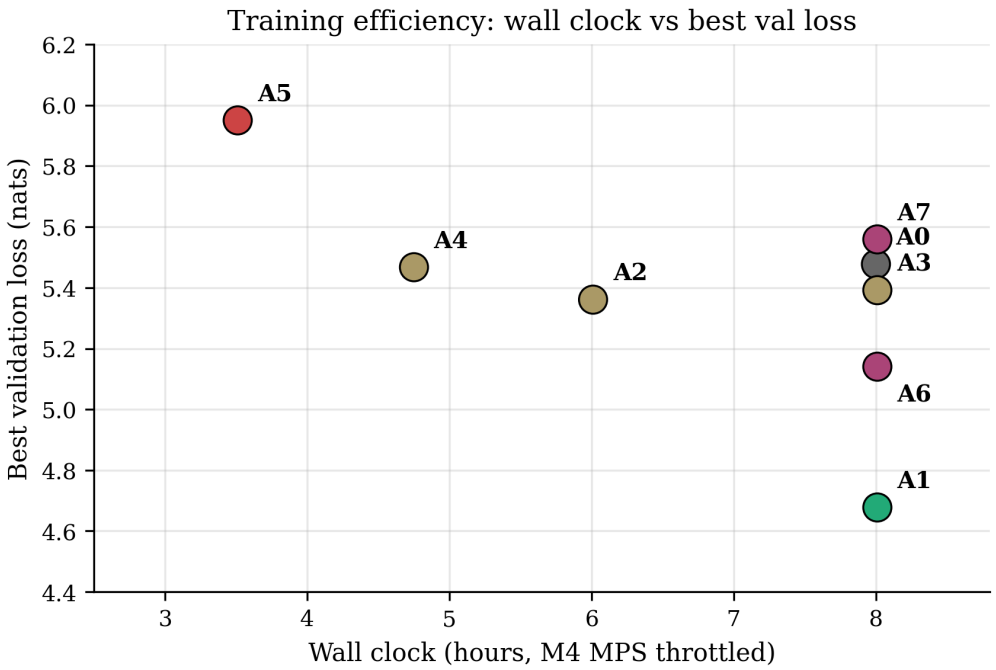

Figure 9. Training efficiency: wall-clock time against best validation loss. A1 dominates the Pareto frontier.

9. Discussion

9.1 Implications for small-LM architecture choice

Practitioners building decoder language models at the 25M parameter scale should adopt RoPE in preference to learned absolute positional embeddings, and should retain AdamW as the optimizer rather than adopting Muon under default hyperparameters. The remaining components of the 2026 consensus stack (RMSNorm, QK-norm, SwiGLU) cannot be recommended for adoption at this scale based on our results, particularly given the destructive interaction observed when they are combined with AdamW. Models targeting deployment efficiency may still prefer RMSNorm for its computational simplicity, with the understanding that the quality gain at this scale is modest and may carry trade-offs with optimizer choice.

9.2 The Muon-norm dependency reframes prior results

Prior literature presenting Muon and the modern normalization stack typically reports them together, attributing the combined performance to the sum of individual component benefits. Our results suggest a different interpretation is possible: at small scale, the apparent quality contribution of RMSNorm and QK-norm may be substantially confounded with their stabilizing effect on Muon. If Muon is the optimizer in use, RMSNorm and QK-norm appear necessary. If AdamW is the optimizer in use, these components contribute only marginal value (less than 0.13 nats individually in our experiments). The same component can have very different roles depending on the surrounding configuration.

Whether this dependency persists at larger scales is an open question. Our results do not directly speak to billion-parameter models. However, the possibility that the consensus stack at scale benefits from a similar functional dependency, with one or two components serving primarily to stabilize others rather than independently improving quality, deserves systematic investigation.

9.3 The reproducibility observation

The 1.63 nat gap between published Apollo v2 (3.95) and reproduced Apollo v2 (5.58) on identical hardware is, in our view, a more important methodological finding than the ablation results themselves. If a 25M parameter model trained on identical code and data can produce validation losses differing by 1.63 nats on the same laptop in adjacent weeks, then small-LM validation losses are not directly comparable across publications without multiple-seed reporting. We recommend that future small-LM papers report a minimum of three seed runs with standard deviation.

10. Limitations

This study has five primary limitations. First, all runs use a single random seed; we have not measured cross-seed variance directly within Apollo-Helios. Second, all runs use a single corpus composition (Apollo v2's 34/35/31 mix). Third, all runs use a single parameter scale (25M body); the destructive interactions in A5 and A7 may not appear at larger scales. Fourth, our wall-clock budget caps each run at 8 hours; longer runs might allow A5 to recover, though the early-stopping trigger suggests not. Fifth, we did not run a learning-rate sweep for Muon; a smaller Muon learning rate might recover competitive performance in the simple-normalization regime, which would qualify but not overturn our Muon-norm dependency finding.

11. Conclusion

We presented a three-stage investigation of decoder transformer training at the 25M parameter scale. Apollo v1 established a 2022-era baseline. Apollo v2 fixed three observed failure modes by replacing the tokenizer and rebalancing the code corpus. Apollo-Helios held v2 fixed and ablated each component of the 2026 consensus stack across eight 8-hour training runs.

The principal finding is that at 25M scale, rotary position embeddings with AdamW account for nearly the entire achievable improvement from the modern stack. The remaining three architectural components contribute essentially nothing individually and interact destructively when combined. The Muon optimizer exhibits a strong functional dependency on RMSNorm and QK-norm: applied without these stabilizers, Muon damages convergence by 0.88 nats. AdamW with RoPE alone remains the dominant configuration.

Beyond the architectural findings, we document a 1.63 nat reproducibility gap between published and re-run Apollo v2 validation losses on identical hardware and code. This gap exceeds the magnitude of every individual component contribution measured in our ablations, suggesting that single-run validation loss is an unreliable evaluation metric at this scale.

All code, training logs, and checkpoints from the Apollo-Helios runs are released at github.com/Rburra1/Apollo-Helios.

References

Allal, L. B., et al. (2025). SmolLM2: When Smol Goes Big. Hugging Face Technical Report.

Biderman, S., et al. (2023). Pythia: A Suite for Analyzing Large Language Models Across Training and Scaling. ICML 2023.

Google DeepMind. (2025). Gemma 3 270M Technical Report. arXiv preprint.

Gunasekar, S., et al. (2023). Textbooks Are All You Need. arXiv:2306.11644.

Henry, A., Dachapally, P. R., Pawar, S., and Chen, Y. (2020). Query-Key Normalization for Transformers. EMNLP Findings.

Jordan, K. (2024). Muon: An Optimizer for Hidden Layers in Neural Networks. Blog post, kellerjordan.github.io.

Karpathy, A. (2022). nanoGPT. GitHub repository, github.com/karpathy/nanoGPT.

Liu, J., et al. (2025). Muon is Scalable for LLM Training. arXiv:2502.16982 (Moonshot AI).

Sellam, T., Yadlowsky, S., Wei, J., et al. (2022). The MultiBERTs: BERT Reproductions for Robustness Analysis. ICLR 2022.

Shazeer, N. (2020). GLU Variants Improve Transformer. arXiv:2002.05202.

Su, J., Lu, Y., Pan, S., Murtadha, A., Wen, B., and Liu, Y. (2021). RoFormer: Enhanced Transformer with Rotary Position Embedding. arXiv:2104.09864.

Zhang, B., and Sennrich, R. (2019). Root Mean Square Layer Normalization. NeurIPS 2019.